Pràctica de cerca local

Denys · Albert · Bernat

2026-10-20

Taula de continguts

1 Introducció **4**

2 Organització de l'equip **5**
2.1 Seguiment del guió i repartiment de tasques 5

Estructura del projecte **6**

3 Context i materials **8**
3.1 Codi propi 8
3.2 Biblioteca del problema 8
3.3 Biblioteca d'algorismes 9
3.4 Dependències de gràfiques 9
3.5 Material de consulta 9
3.6 Com es connecta tot (compilació i execució) 10

4 Anàlisi del problema **11**
4.1 Definició del problema 11
4.2 Mida de l'espai de solucions 12
4.3 Observacions sobre la representació i els criteris de validesa 12

5 Espai de solucions **13**
5.1 Solucions i estats inicials 13
5.2 Estructura i qualitat dels estats 14
5.3 Construcció dels estats en el codi 14
5.4 Operadors i definició del veïnatge 15

6 Experiments **17**
6.1 Heurístiques 18

7 Pipeline d'execució **20**
7.1 Entrada i preparació de l'experiment 20
7.2 Muntatge de la cerca 21
7.3 Expansió i avaluació dels estats 21
7.4 Biblioteques i sortida 22

8 Resultats experimentals **23**
8.1 Operadors i mida del veïnatge 23
8.2 Estratègia de solució inicial 24
8.3 Paràmetres de Simulated Annealing 25
8.4 Escalabilitat amb el nombre de clients i centrals 27
8.5 Penalització dels clients garantits no servits 28
8.6 Ampliació del parc de centrals 29

8.7 Comparació de les heurístiques . . . 30
8.8 Hill Climbing i Simulated Annealing . . . 32

9 Execució i reproducció **33**
9.1 Requisits i directori de treball . . . 33
9.2 Compilació . . . 33
9.3 Comprovació de la representació . . . 33
9.4 Execució dels experiments . . . 34
9.5 Generació de figures i comparacions . . . 34
9.6 Generació del report . . . 34

10 Anàlisi i discussió **35**
10.1 Importància dels operadors . . . 35
10.2 Efecte de la solució inicial . . . 35
10.3 Paper de les heurístiques . . . 35
10.4 Comparació dels algoritmes . . . 36
10.5 Qualitat i cost computacional . . . 36

11 Conclusions **37**

12 Treball d'innovació **38**
12.1 Tema escollit . . . 38
12.2 Repartiment del treball . . . 38
12.3 Referències trobades . . . 38
12.4 Dificultats trobades . . . 38

Capítol 1

Introducció

Informe de la primera practica de l'assignatura Inteligencia Artificial (GCED-UPC).

Capítol 2

Organització de l'equip

- **Integrants:** Denys, Albert i Bernat.
- **Mecanismes de comunicació:**
- **Calendari de reunions i acords presos en cadascuna:**
- **Com s'han resolt les discrepàncies:**

2.1 Seguiment del guió i repartiment de tasques

Setmana	Objectius del guió	Responsable	Estat
1	Entendre AIMA, executar els exemples, llegir l'enunciat		
2	Dissenyar la representació de l'estat i la seva interfície		
3	Implementar estat, solució inicial i operadors		
4	Generadors de successius, estat final, heurístiques		
5	Planificar, executar i analitzar els experiments		
6	Documentació final		

Estructura del projecte

El repositori es divideix en carpetes segons la funció que desenvolupen:

```
repo_PRAC1/
  src/
    IA/PracticaEnergia/   Codi propi de la pràctica.
  lib/
    *CentralEnergia.jar*    Biblioteca del problema energètic.
    *AIMA.jar*              Biblioteca d'algorismes de cerca.
    *lib/*                  Dependències distribuïdes amb AIMA.
  docs/
    *aima-src/*             Codi font d'AIMA per a consulta.
    *javadoc/*              Documentació de CentralEnergia.jar.
  results/                 Resultats dels experiments i figures.
  report/                  Font Quarto i fitxers de l'informe.
  out/                     Classes compilades de Java.
  *README.md*              Instruccions d'ús i reproducció.
  *.gitignore*             Regles de Git per al repositori.
```

La carpeta `src/IA/PracticaEnergia/` conté el **codi desenvolupat pel grup**. Els fitxers més importants són:

- `EstadoEnergia.java`: representa un estat i implementa les operacions sobre les assignacions de clients a centrals.
- `GeneradorSucesoresHC.java` i `GeneradorSucesoresSA.java`: generen, respectivament, els veïns de Hill Climbing i un successor aleatori per a Simulated Annealing.
- `FuncionHeuristica.java`: defineix les funcions que avaluen la qualitat dels estats.
- `Ejecucion.java`: connecta el problema amb els algorismes d'AIMA i recull les mètriques de cada execució.
- `MainExperimentos.java`: executa la bateria d'experiments i desa els resultats en fitxers CSV.
- `Graficas.java`: llegeix els CSV i genera les figures del report.
- `Comparaciones.java`: realitza les comparacions estadístiques entre configuracions.
- `PruebaEstado.java`: comprova la coherència de la representació, els operadors i les solucions inicials.

`lib/` i `docs/` són **material extern** que no forma part del codi desenvolupat pel grup.

La carpeta `lib/` conté les **biblioteques necessàries per compilar i executar el projecte**. `docs/` conserva el **material de consulta**, `results/` conté els **resultats generats pels**

experiments i `report/` conté la **font i el PDF de l'informe**. La carpeta `out/` és un directori generat durant la compilació i conté els fitxers `.class`; no és codi font.

Capítol 3

Context i materials

En aquest apartat es descriuen les dependències i la manera com es connecten entre si els components del projecte.

3.1 Codi propi

src/

El codi propi, que es troba a la carpeta `src/`, està organitzat en el paquet **`IA.PracticaEnergia`**.

Aquest paquet conté la **representació del problema** (EstadoEnergia), els **operadors i generadors de successors**, les **funcions heurístiques**, l'**execució dels algoritmes**, els **experiments** i la **generació de gràfiques**.

3.2 Biblioteca del problema

lib/CentralEnergia.jar/

La biblioteca del problema està formada per les classes i funcions necessàries per resoldre el problema de distribució d'energia elèctrica.

El fitxer `lib/CentralEnergia.jar`, proporcionat amb l'enunciat, conté les classes de suport agrupades en el paquet **`IA.Energia`**. En concret:

- `Central` i `Centrales` representen les centrals elèctriques.
- `Cliente` i `Clientes` representen els clients, el seu consum i el tipus de contracte.
- `VEnergia` proporciona constants i funcions del model, com els costos de les centrals i les pèrdues de transport.

El paquet `IA.Energia` proporciona, per tant, les dades i les fórmules bàsiques del problema. El paquet propi `IA.PracticaEnergia` les utilitza per construir `EstadoEnergia`, precalcular les magnituds del problema i comprovar si una assignació respecta les capacitats i els contractes garantits. Per exemple:

```
import IA.Energia.Central;
import IA.Energia.Centrales;
import IA.Energia.Cliente;
import IA.Energia.Clientes;
import IA.Energia.VEnergia;
```

La implementació de `IA.Energia` no forma part del codi del grup: es carrega des de `CentralEnergia.jar`. La seva API es pot consultar a `docs/javadoc/`.

3.3 Biblioteca d'algorismes

lib/AIMA.jar

`lib/AIMA.jar` proporciona els **algorismes i les interfícies generals de cerca** d'AIMA. Entre les classes que utilitza el projecte hi ha:

- `Problem`, `SearchAgent` i `Successor`, que permeten definir i executar un problema de cerca.
- `SuccessorFunction`, `GoalTest` i `HeuristicFunction`, que són les interfícies que implementa o utilitza el codi propi.
- `HillClimbingSearch` i `SimulatedAnnealingSearch`, que són els dos algoritmes de cerca local emprats.

Aquestes classes pertanyen principalment als paquets `aima.search.framework` i `aima.search.informed`. AIMA proporciona la infraestructura i els algoritmes, però la representació de l'estat, els operadors i les funcions heurístiques concretes de la pràctica s'implementen a `IA.PracticaEnergia`.

3.4 Dependències de gràfiques

lib/lib/

La classe `Graficas` (del paquet `IA.PracticaEnergia`) llegeix els fitxers CSV dels experiments i genera les figures PNG del report.

Per fer-ho utilitza **JFreeChart**, principalment els paquets `org.jfree.chart` i `org.jfree.data`.

Els JAR necessaris es troben a `lib/lib/`, una carpeta amb dependències d'AIMA que s'ha mantingut dins del repositori per a l'execució del projecte. En particular:

- `jfreechart-1.0.13.jar` conté la biblioteca de gràfiques.
- `jcommon-1.0.16.jar` conté classes comunes que JFreeChart necessita.

Dins de `lib/lib` també hi ha altres dependències distribuïdes amb AIMA, com `iText`, `JUnit` i components gràfics, però no totes són necessàries per executar aquesta pràctica.

La carpeta es conserva perquè `AIMA.jar` declara part d'aquestes dependències amb rutes relatives `lib/...` i perquè `Graficas.java` necessita JFreeChart per generar les figures.

Durant l'execució de `Graficas` aquestes classes es carreguen juntament amb el codi propi.

3.5 Material de consulta

docs/

`docs/` conté **material de consulta** que no forma part del codi desenvolupat pel grup:

- `docs/aima-src/` és el **codi font d'AIMA** lliurat com a material docent i **consultat per entendre el comportament dels algorismes**. Serveix per consultar com funcionen internament les classes utilitzades pel projecte, especialment `HillClimbing`

i `SimulatedAnnealing`. També inclou **exemples i demostracions docents**, en les subcarpetes `Demos/`, `IA` i `aima`.

- `docs/javadoc/` conté **documentació HTML de l'API de `CentralEnergia.jar`** generada amb Javadoc i conservada com a material de consulta. La documentació principal correspon al paquet `IA.Energia` i inclou `Central.html`, `Centrales.html`, `Cliente.html`, `Clientes.html` i `VEnergia.html`. També conté els recursos auxiliars per navegar per la documentació HTML.

Les carpetes anteriors serveixen per consultar les interfícies i les implementacions de suport, però no formen part del codi desenvolupat pel grup i no es modifiquen.

3.6 Com es connecta tot (compilació i execució)

(1) **Durant la compilació**, els JAR que contenen les dependències principals (`CentralEnergia.jar` i `AIMA.jar`) s'afegeixen al *classpath*:

```
javac -Xlint:all,-path -cp "lib\CentralEnergia.jar;lib\AIMA.jar" `
  -d out (Get-ChildItem -Recurse src -Filter *.java | % FullName)
```

Això permet que el compilador resolgui les classes dels paquets **`IA.Energia`** i **`aima.search`**. El resultat del codi propi es desa a `out/`.

(2) **Durant l'execució**, `out/` i les biblioteques s'inclouen al *classpath*:

```
java -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.MainExperimentos smoke
```

Les classes de **`IA.PracticaEnergia`** es carreguen des de `out/`, mentre que les classes de **`IA.Energia`**, `aima.search` i `org.jfree` es carreguen des dels fitxers JAR.

No cal descomprimir-los: el *classpath* indica a Java on ha de trobar les classes compilades.

Capítol 4

Anàlisi del problema

4.1 Definició del problema

El problema consisteix en la planificació de l'assignació de clients a centrals d'energia, amb l'objectiu de maximitzar el benefici net. Això implica prendre decisions sobre quins clients assignar a quines centrals, tenint en compte les restriccions de capacitat i els costos associats.

La descripció formal i completa del problema es troba a l'apartat 4 del document `PracticaBusqueda-local.pdf`. Aquí destacarem únicament els punts més rellevants per entendre la implementació i els experiments realitzats.

Una solució assigna a cada client **com a màxim una** central, o cap si és un **client no garantitzat** al qual es decideix no servir. El criteri a maximitzar és

benefici = ingressos dels clients servits−cost de les centrals (activa si tenen algun client, parada si no)−inden

Hi ha dues propietats de l'enunciat que condicionen tot el disseny i que convé tenir presents perquè expliquen gairebé tots els resultats experimentals:

1. **Una central en marxa costa tota la seva producció, s'usi o no.**

El cost no depèn de quanta energia es serveixi des d'ella, només de si està engegada. És a dir, **el cost marginal d'afegir un client a una central ja engegada és exactament zero**.

2. **Les pèrdues de transport no són un cost en euros, són un consum de capacitat.**

Al client se li factura el seu consum contractat; el que canvia amb la distància és quants Mw cal produir: `mw = consum / (1 - pèrdua(distància))`, fins a 2,5 vegades el consum si la central està a més de 75 km.

De la primera propietat se'n dedueix una cosa que resulta decisiva: **el benefici no depèn de com es reparteixin els clients, sinó només de quines centrals estan enceses i de quins clients queden sense servir.**

El repartiment concret només importa de manera indirecta, perquè determina si el repartiment és possible. Ho desenvoluparem a l'apartat **Espai de solucions**.

És un **problema de cerca local**, i no de cerca en un espai d'estats clàssic, per tres raons.

1. No existeix un estat objectiu reconeixible: **qualsevol assignació vàlida és una solució** i el que es busca és la millor, no una concreta.

2. **L'espai és inabastable** (vegeu **Mida de l'espai de solucions**), així que no es pot explorar de manera sistemàtica ni limitar amb una heurística admissible.
3. **I el camí fins a la solució no té cap valor**: només interessa l'estat final, no la seqüència d'operadors que hi porta.

4.2 Mida de l'espai de solucions

Cada client pot estar en qualsevol de les `nCentrals` centrals o sense servir, així que hi ha de l'ordre de `(nCentrals + 1)^nClients` combinacions, que en l'escenari base són `41^1000`, un nombre d'unes 1.600 xifres.

Només una fracció compleix les restriccions de capacitat i de servei garantit, però continua sent una **fracció immensament gran**.

Convé contrastar-ho amb la dimensió que de veritat determina el benefici. Pel que s'ha dit a **Definició del problema**, el benefici només depèn de **quines centrals estan enceses** (40 variables binàries, `2^40` combinacions) i de **quins no garantits es serveixen**.

L'espai d'assignacions és enormement més gran que l'espai de decisions que afecten l'objectiu, i aquesta desproporció és exactament la que produeix les mesetes que s'expliquen a **Espai de solucions**.

4.3 Observacions sobre la representació i els criteris de validesa

Abans de definir la representació interna, convé distingir entre un **estat** i una **solució vàlida**. Un estat és una assignació concreta dels clients a les centrals, mentre que **és solució vàlida quan compleix totes les restriccions del problema**. En particular:

- Cada client pot estar assignat a una única central o quedar sense servir;
- Cap central pot superar la seva capacitat de producció, tenint en compte les pèrdues de transport;
- Tots els clients amb contracte garantit han de rebre servei;
- Els clients no garantits poden quedar sense assignar, però això redueix el benefici perquè cal pagar-ne la indemnització.

Per tant, **no tots els estats del domini combinatori descrit a "Mida de l'espai de solucions" són factibles**. Aquests criteris de validesa s'especifiquen internament en el codi del projecte.

La representació i els operadors descarten les assignacions que superen la capacitat d'una central i, en el funcionament normal, impedeixen desassignar clients garantits.

Aquesta elecció incorpora les **restriccions** en la generació de successors i evita que els algoritmes hagin d'explorar una gran quantitat d'estats impossibles.

Capítol 5

Espai de solucions

Després d'haver definit el problema i d'haver analitzat la seva mida, aquest apartat concreta com es representa i s'explora l'espai de solucions en el programa.

Es descriuen els estats inicials, l'estructura i la qualitat dels estats, la seva construcció en el codi i els operadors que defineixen el veïnatge.

5.1 Solucions i estats inicials

La cerca local necessita un estat inicial, però no parteix d'una solució òptima.

El projecte implementa dues estratègies principals:

- **ALEATORIA**: recorre els clients garantits en un ordre aleatori i els assigna a una de les vuit centrals més properes que encara tenen capacitat.

Si cap d'aquestes no hi cap, busca una altra central amb capacitat per no incomplir el servei garantit. Els clients no garantits comencen sense servir.

Aquesta estratègia permet estudiar l'efecte de la posició inicial, però pot encendre més centrals o aprofitar pitjor la capacitat disponible.

- **VORAZ**: ordena primer els clients garantits segons el consum i intenta reutilitzar les centrals ja enceses. Només obre una central nova quan és necessari, i posteriorment assigna clients no garantits a l'espai que queda lliure. Com que el cost d'una central encesa ja s'ha pagat, afegir-hi un client que hi cap no té cost marginal de producció i pot augmentar el benefici.

Ambdues estratègies **prioritzen les centrals més properes**. Això redueix les pèrdues de transport i evita que una assignació aleatòria consumeixi tota la capacitat disponible abans d'haver servit els clients garantits.

La llavor del generador aleatori es fixa per poder reproduir exactament una solució inicial aleatòria.

La classe `PruebaEstado` comprova que les dues construccions són coherents i que serveixen tots els clients garantits.

També verifica que la solució voraz obté un benefici superior a la solució aleatòria en l'escenari de prova.

Això no implica que la solució voraz sigui òptima: només proporciona un punt de partida de més qualitat per a la cerca local.

5.2 Estructura i qualitat dels estats

Una assignació concreta pot tenir molts estats equivalents des del punt de vista del benefici. Si dos estats serveixen els mateixos clients i mantenen enceses les mateixes centrals, el benefici és el mateix encara que alguns clients estiguin distribuïts de manera diferent entre aquestes centrals.

Les distàncies sí que poden canviar les pèrdues i la capacitat disponible, però no constitueixen directament un cost monetari: afecten la factibilitat perquè determinen els megawatts que cal produir.

Aquesta propietat crea **mesetes** en la funció objectiu. Per exemple, moure un client entre dues centrals que ja estan enceses pot no modificar ni els ingressos ni el cost de les centrals.

Un Hill Climbing basat només en el benefici no considera aquest moviment una millora i pot aturar-se encara que un conjunt de moviments posteriors permetés apagar una central.

Per això són rellevants els operadors d'assignació i de gestió de centrals, així com les heurístiques que penalitzen les pèrdues o la capacitat ociosa.

Finalment, **els estats es poden comparar** mitjançant el benefici, però també mitjançant mesures auxiliars: nombre de clients servits, clients garantits no servits, centrals enceses, megawatts perduts i capacitat desaprofitada.

Aquestes magnituds permeten analitzar si una millora del benefici s'obté servint més clients, reduint costos d'activació o aprofitant millor la capacitat.

5.3 Construcció dels estats en el codi

src/IA/PracticaEnergia/EstadoEnergia.java

La classe `EstadoEnergia` és la representació concreta de cada estat. La **part variable** es manté amb tres vectors:

- `asignacion`, indexat pel client, conté l'identificador de la central on està assignat o `-1` (`SIN_ASIGNAR`) si no rep servei;
- `ocupacion` indica els megawatts de producció compromesos a cada central, incloent-hi les pèrdues de transport;
- `clientesPorCentral` compta els clients de cada central i permet saber si està encesa.

La resta de camps dinàmics, com el benefici, el nombre de centrals enceses, els megawatts produïts i els clients garantits sense servir, també es mantenen de manera incremental.

Així, quan es crea un successor, el constructor de còpia clona els tres vectors i els valors numèrics, i l'operador només actualitza les diferències produïdes. No cal reconstruir ni recórrer tota l'assignació per recalcular el benefici.

Les dades que no canvien entre estats es precalculen una sola vegada a `inicializaEscenario`: costos i produccions de les centrals, ingressos i penalitzacions dels clients, i els megawatts necessaris per servir cada parella client-central.

També es guarden les `k` centrals més properes a cada client i els `k` clients més propers, que limiten el veïnatge. Aquesta separació redueix la memòria necessària per estat i fa que els successors siguin independents del seu estat pare. (Veure apartat *Operadors i definició del veïnatge* per més detalls sobre el paràmetre `k`.)

El constructor inicial parteix de tots els clients sense servir i totes les centrals parades.

En aquest punt el benefici inclou el cost de mantenir les centrals parades i, si escau, les indemnitzacions.

A continuació aplica `ALEATORIA`, `VORAZ` o `VACIA`.

Aquesta última només s'utilitza en el mode experimental que permet penalitzar el servei garantit no satisfet.

5.4 Operadors i definició del veïnatge

src/IA/PracticaEnergia/ConjuntoOperadores.java src/IA/PracticaEnergia/GeneradorSucesoresHC.java src/IA/PracticaEnergia/GeneradorSucesoresSA.java

Un **operador** transforma un estat en un estat veí sempre que el moviment sigui legal.

El projecte implementa els operadors següents:

- **MOVER**: canvia un client servit a una altra central propera si hi ha capacitat. Pot reduir les pèrdues, però normalment no canvia el benefici si les dues centrals ja estaven enceses.
- **ASIGNAR/DESASIGNAR**: dona o retira el servei d'un client. En el mode normal només es poden desassignar clients no garantits; en el mode de penalització també es poden deixar sense servir clients garantits.
- **INTERCAMBIAR**: intercanvia les centrals de dos clients servits. La comprovació calcula la capacitat disponible després de retirar els dos clients, de manera que l'intercanvi es fa de forma simultània i no genera estats intermedis impossibles.
- **ABRIR**: encén una central parada i hi assigna tants clients no servits com permet la seva capacitat, prioritzant la densitat de benefici per megawatt.
- **CERRAR**: apaga una central i intenta reubicar tots els seus clients en altres centrals. Si algun client no hi cap, el successor es descarta.

Els dos darrers són operadors compostos. Són necessaris perquè un moviment individual pot deixar el benefici inalterat, mentre que buidar una central permet estalviar-ne el cost d'activació en una sola transició. Per evitar duplicats, l'intercanvi només es genera una vegada per cada parella de clients.

Els operadors compostos són importants perquè Hill Climbing només accepta una transició quan millora estrictament el criteri. Si cal moure molts clients per deixar una central buida i poder estalviar-ne el cost, els moviments individuals intermedis poden tenir exactament el mateix benefici i formar una meseta. `ABRIR` i `CERRAR` fan el canvi rellevant en una sola transició i permeten que la cerca detecti directament la millora completa. No són una heurística diferent, sinó una manera més expressiva de definir el veïnatge.

Les **comprovacions** es fan abans de crear el successor i incorporen la capacitat de la central i la restricció dels clients garantits.

La capacitat es comprova amb

$$\text{ocupació actual} + \text{Mw necessaris per al client} \leq \text{producció de la central.}$$

Respecte del codi:

*Vegeu **Pipeline d'execució** per a més detalls sobre la implementació del codi.*

`ConjuntoOperadores` no implementa les transformacions individualment, sinó que **defineix**

els conjunts d'operadors disponibles (`MOVER`, `MOVER_ASIGNAR`, `MOVER_INTERCAMBIAR`, `MOVER_ASIGNAR_CENTRALES` i `COMPLETO`) i indica quins tipus de moviment inclou cada conjunt.

Les classes `GeneradorSucesoresHC` i `GeneradorSucesoresSA` **utilitzen aquesta configuració per construir els successors**.

`GeneradorSucesoresHC`, per la seva part, enumera tots els successors legals dels operadors seleccionats i els entrega a Hill Climbing. Per tant, el nombre de successors depèn del conjunt triat i del valor de `k`.

En canvi, `GeneradorSucesoresSA` escull aleatòriament un operador i els seus paràmetres i retorna un únic successor. Fa fins a 30 intents per trobar un moviment legal; si no en troba cap, retorna una còpia de l'estat actual.

Per tant, el mateix espai de solucions es recorre amb veïnats diferents segons l'algoritme: **complet per a Hill Climbing** i **mostrejat aleatòriament per a Simulated Annealing**.

El paràmetre `k` limita aquest veïnatge a les `k` centrals més properes a cada client i als `k` clients més propers quan l'operador ho requereix. Amb `k` petit es redueix el nombre de successors, el temps d'avaluació i la memòria necessària; amb `k` gran s'exploren més alternatives, però el cost computacional augmenta. No es tracta d'un límit arbitrari sobre la capacitat: només restringeix quines parelles client-central es consideren en els moviments.

En l'escenari estudiat, les centrals llunyanes solen ser menys atractives perquè les pèrdues de transport consumeixen més capacitat. Per això es comparen diversos valors de `k` i es fixa un valor moderat quan l'ampliació del veïnatge ja no aporta una millora clara del benefici.

Capítol 6

Experiments

En aquest projecte, **un experiment és una configuració concreta del problema que es repeteix sota una llavor determinada (valors inicials específics) i amb una mesura de qualitat ben definida**.

En termes pràctics, un experiment és el conjunt d'eleccions que fixen el problema: *nombre de clients, nombre i tipus de centrals, proporció de clients garantits, estratègia inicial, conjunt d'operadors, mida del veïnatge* `k`*, algoritme de cerca i paràmetres associats.*

Això és fonamental perquè el problema d'assignació energètica té una **estructura complexa** i no és suficient provar una sola solució.

La qualitat del resultat depèn de molts components simultanis: *la distribució geogràfica dels clients, la capacitat de les centrals, les pèrdues de transport, la capacitat de servir els clients garantits i el tipus de moviment que permet el veïnatge.*

Per aquest motiu **el projecte està pensat per a una bateria d'experiments**.

Els **experiments permeten separar** quina part del rendiment depèn de la representació de l'estat, quina del veïnatge, quina de la funció heurística i quina de l'algoritme.

D'una banda, això fa possible **comparar solucions** sota les mateixes condicions; de l'altra, permet **detectar** si el problema està condicionat per una meseta, per la capacitat ociosa o per la necessitat de tancar centrals.

Les **variacions principals** que es comparegen són:

- **solució inicial**: `ALEATORIA` i `VORAZ`;
- **operadors**: moviments simples, assignació, intercanvi i obertura/tancament de centrals;
- **veïnatge**: valor de `k` que limita els successors possibles;
- **algoritme**: `Hill Climbing` i `Simulated Annealing`;
- **heurística**: benefici pur, penalització de pèrdues i penalització de capacitat ociosa;
- **escenari**: nombre de clients i centrals, i proporció de clients garantits.

Cada configuració es prova amb **10 rèpliques**. Les rèpliques de totes les configuracions que es comparen utilitzen el mateix conjunt de llavors; per tant, en una rèplica concreta, els mètodes parteixen del mateix escenari i, quan correspon, de la mateixa solució inicial.

Això permet fer comparacions emparellades i fa que les diferències observades s'atribueixin a la configuració avaluada i no a un escenari aleatori diferent.

Per a les comparacions implementades a `Comparaciones` es compta, rèplica a rèplica, quantes vegades una configuració obté un benefici superior a l'altra.

Sota la hipòtesi nul · la que totes dues configuracions tenen la mateixa probabilitat de guanyar, el nombre de victòries segueix una distribució binomial $B(n, 0{,}5)$.

El contrast és bilateral i es considera significativa una diferència quan $p < 0{,}05$. Els empats no compten com una victòria i es descarten del nombre de rèpliques decisives del contrast.

Aquest contrast s'utilitza per a les comparacions de benefici entre conjunts d'operadors, estratègies de solució inicial, heurístiques i algoritmes de cerca.

Per a mètriques com el temps de còmput o els nodes expandits, el report mostra els valors resumits, però no s'hi aplica automàticament aquest contrast de victòries.

El **resultat de cada experiment** es resumeix amb mètriques com el benefici mitjà, la validesa de la solució, el temps de còmput, el nombre de nodes expandits i la qualitat del servei.

En conjunt, **els experiments mostren que la decisió més determinant no és tant el cercador com el disseny dels operadors i la qualitat de la solució inicial**.

Després de l'anàlisi dels resultats, es conclou: *la combinació més equilibrada és la que reutilitza la inicial voraz, empra operadors compostos i limita el veïnatge a un valor moderat, perquè millora el benefici sense explotar excessivament el temps de càlcul.*

6.1 Heurístiques

FuncionHeuristica.java

La **funció heurística** és el criteri que utilitza l'algoritme de cerca local per comparar l'estat actual amb els seus successors.

En aquest problema no cal estimar la distància fins a un estat objectiu, perquè no hi ha un objectiu únic: qualsevol assignació vàlida pot ser una solució.

Per tant, la heurística ha de prioritzar els estats que poden donar un benefici final més alt i, alhora, ha de guiar la cerca quan diversos estats tenen exactament el mateix benefici.

El criteri principal és el **benefici real del problema**:

$B(s) =$ ingressos dels clients servits−cost de les centrals enceses−indemnitzacions dels clients no garantits no

Les **restriccions de capacitat i d'assignació** s'apliquen en generar els successors, de manera que no cal afegir-les com a penalitzacions a la funció heurística en el funcionament normal.

Els clients garantits tampoc no es poden desassignar després de la construcció de l'estat inicial. Això permet que la funció se centri en ordenar els estats factibles segons la seva qualitat.

El projecte implementa tres variants:

Heurística	Expressió del criteri de qualitat	Funció
H1: benefici	$B(s)$	Optimitza directament l'objectiu del problema.
H2: penalització de pèrdues	$B(s) - w \cdot Mw_{\text{perduts}}(s)$	Prefereix assignacions que necessiten produir menys energia perduda en el transport.
H3: penalització de capacitat ociosa	$B(s) - w \cdot Mw_{\text{ociosos}}(s)$	Prefereix concentrar la demanda en centrals enceses i amb menys capacitat sense utilitzar.

En **H2**, les pèrdues no són un cost monetari directe: són megawatts addicionals que cal produir i, per tant, redueixen la capacitat disponible.

La penalització de les pèrdues permet donar una puntuació més precisa als estats, especialment en situacions on la distància entre clients i centrals és important.

Penalitzar-les proporciona, però, un criteri de desempat útil quan un moviment canvia la distància d'un client sense encendre ni apagar cap central.

En **H3**, la capacitat ociosa és la producció disponible de les centrals enceses que no s'està utilitzant.

La penalització afavoreix solucions més compactes i pot facilitar que una central quedi buida i es pugui apagar en moviments posteriors.

Tant H2 com H3 són criteris secundaris: el benefici que es reporta als experiments és sempre el benefici real $B(s)$, no el valor penalitzat de la heurística.

El paràmetre w expressa el pes del criteri secundari en euros equivalents per megawatt. A l'experiment 7 es proven els pesos 1, 10, 50 i 200 €/MW, tant amb operadors simples (`MOVER_ASIGNAR`) com amb operadors compostos (`MOVER_ASIGNAR_CENTRALES`).

El pes té unitats d'euros per megawatt i permet interpretar el terme secundari en la mateixa escala que el benefici. Ha de ser prou gran per distingir estats amb el mateix benefici, però prou petit per no invertir l'ordre entre dos estats amb beneficis reals diferents. Amb un pes massa alt, la cerca pot prioritzar reduir pèrdues o capacitat ociosa encara que això empitjori el benefici real; per això l'experiment 7 prova un rang de pesos i les configuracions sempre es comparen amb el benefici real final.

Les **heurístiques secundàries** són especialment importants amb operadors simples.

El benefici és constant quan es mou un client entre dues centrals que ja estan enceses, i Hill Climbing interpreta aquests moviments com una meseta. H2 pot trencar-la perquè les pèrdues depenen de la distància, mentre que H3 pot afavorir una redistribució que permeti tancar una central.

Amb operadors compostos, un únic moviment pot fer explícit el canvi d'obrir o tancar centrals; en aquest cas H1 acostuma a ser suficient i les variants serveixen sobretot per analitzar el comportament de la cerca.

Finalment, AIMA espera una funció que s'ha de **minimitzar**, mentre que el problema es formula com una maximització. Per això `FuncionHeuristica` calcula primer el criteri anterior i retorna el seu negatiu.

La qualitat que apareix als CSV i a les figures, en canvi, és sempre el benefici real de l'estat final.

Com que l'estat manté incrementalment el benefici, les pèrdues i la capacitat ociosa, l'avaluació de qualsevol de les tres variants es fa en temps constant $O(1)$.

Capítol 7

Pipeline d'execució

Abans d'analitzar els algorismes i els resultats, és útil veure **com s'encadenen els components del projecte durant una execució**.

La bateria d'experiments no implementa directament la cerca: prepara les dades, crea l'estat inicial i delega la cerca a les classes d'AIMA.

El **flux general** és:

MainExperimentos.java inicialitza l'escenari crea l'estat inicial crida **Ejecucion.java** crea el generador de successors crea el problema de cerca amb **Problem** (AIMA) crea l'agent amb **SearchAgent** (AIMA) crea **HillClimbingSearch** o **SimulatedAnnealingSearch** (AIMA) executa **HillClimbingSearch** o **SimulatedAnnealingSearch** demana successors a **GeneradorSucesoresHC.java** o **GeneradorSucesoresSA.java** aplica operadors sobre còpies de **EstadoEnergia.java** avalua els estats amb **FuncionHeuristica.java** comprova el criteri de finalització amb **EstadoFinal.java** recull el resultat i l'escriu a **results/expN.csv**

7.1 Entrada i preparació de l'experiment

L'execució comença en el mètode `main` de `MainExperimentos`.

L'argument de la línia d'ordres **selecciona un experiment concret** (`1` a `8`), tots els experiments (`todos`) o una comprovació ràpida (`smoke`).

Abans d'executar cada rèplica, `MainExperimentos` **fixa la configuració de l'escenari base**: nombre i tipus de centrals, nombre de clients, proporció de clients garantits i mida del veïnatge `k`.

La funció `escenarioBase` delega la inicialització a `EstadoEnergia.inicializaEscenario`.

Aquesta crida carrega les centrals i els clients des de la biblioteca del problema, calcula les dades que es mantindran constants durant la cerca i precalcula les distàncies, les pèrdues i els megawatts necessaris per a cada assignació client-central.

La llavor de la rèplica s'utilitza per fer reproduïbles tant l'escenari com les decisions aleatòries.

Després es crea l'estat inicial, per exemple amb l'estratègia `VORAZ`:

```
EstadoEnergia inicial = new EstadoEnergia(
        EstadoEnergia.EstrategiaInicial.VORAZ, semilla);
```

Aquest objecte **conté l'assignació inicial i totes les magnituds incrementals que**

necessitaran els operadors i les heurístiques.

7.2 Muntatge de la cerca

La classe `Ejecucion` centralitza la diferència entre els dos algorismes.

En una execució de Hill Climbing crea un `HillClimbingSearch` i un `GeneradorSucesoresHC`.

En una execució de Simulated Annealing crea un `SimulatedAnnealingSearch`, amb els paràmetres de temperatura i iteracions, i un `GeneradorSucesoresSA`, inicialitzat amb la llavor corresponent.

Tot seguit, `Ejecucion` **construeix el problema d'AIMA**:

```
Problem problema = new Problem(
        inicial, sucesores, new EstadoFinal(), heuristica);
```

Aquest objecte **agrupa els quatre elements que necessita la infraestructura de cerca**:

- L'estat inicial;
- La funció de successors, que defineix el veïnatge;
- El test de finalització;
- La funció heurística que permet comparar estats.

En aquest problema `EstadoFinal` retorna sempre `false`, perquè **no es busca un estat objectiu concret**.

Hill Climbing acaba **quan no troba cap successor estrictament millor**, mentre que Simulated Annealing acaba **quan consumeix el nombre d'iteracions configurat**.

7.3 Expansió i avaluació dels estats

`Ejecucion` entrega el `Problem` i l'algorisme a `SearchAgent`, que forma part de `AIMA.jar`.

AIMA controla el bucle de cerca i, en cada expansió, demana successors a la funció corresponent.

`GeneradorSucesoresHC` recorre tots els clients, centrals i operadors seleccionats en `ConjuntoOperadores`. Per cada moviment legal crea una còpia d'`EstadoEnergia`, aplica l'operador i incorpora el successor a la llista.

Hill Climbing avalua tota aquesta llista i escull el millor successor segons la heurística.

`GeneradorSucesoresSA` no materialitza tot el veïnatge. Escull aleatòriament un operador i els seus paràmetres, prova fins a 30 combinacions i retorna el primer successor legal.

Si no en troba cap, retorna una còpia de l'estat actual perquè el bucle de Simulated Annealing pugui continuar.

En tots dos casos, la lògica concreta de les transformacions és a `EstadoEnergia`: comprova la capacitat, manté la restricció dels clients garantits i actualitza incrementalment el benefici, l'ocupació i les magnituds energètiques.

`FuncionHeuristica` llegeix aquestes magnituds per calcular el benefici pur o les variants que penalitzen les pèrdues i la capacitat ociosa.

7.4 Biblioteques i sortida

El motor genèric de cerca prové d'`AIMA.jar`.

Les interfícies `Problem`, `SearchAgent`, `SuccessorFunction`, `GoalTest` i `HeuristicFunction`, així com les implementacions `HillClimbingSearch` i `SimulatedAnnealingSearch`, són proporcionades per aquesta biblioteca.

Les dades físiques del problema provenen de `CentralEnergia.jar`, que aporta les classes `Central`, `Centrales`, `Cliente`, `Clientes` i `VEnergia`. El codi de la pràctica les utilitza per construir l'escenari i obtenir costos, consums i pèrdues de transport.

Quan AIMA finalitza la cerca, `Ejecucion` recupera l'estat final i construeix un objecte `Ejecucion.Resultado` amb el benefici, el temps, els nodes expandits, la validesa, els clients servits, les centrals enceses i les magnituds energètiques. `MainExperimentos` agrega aquests resultats, mostra els resums per terminal i els desa a `results/expN.csv`.

Posteriorment, `Graficas` utilitza aquests CSV per generar les figures i `Comparaciones` calcula les comparacions estadístiques.

Capítol 8

Resultats experimentals

Aquest apartat presenta els resultats de la bateria d'experiments descrita a **Experiments**. Cada configuració s'executa amb 10 rèpliques i les configuracions que es comparen comparteixen les mateixes llavors. Les figures es generen a partir dels fitxers CSV de `results/`, i les comparacions de benefici es fan rèplica a rèplica.

Les mètriques principals són el **benefici final**, el **temps de còmput**, el nombre de **nodes expandits**, el nombre de **centrals enceses** i els **clients servits**. En els experiments on és rellevant també s'analitzen les pèrdues de transport, la capacitat ociosa i la validesa de la solució. Els valors agregats es presenten com a mitjana i desviació típica sobre les rèpliques.

8.1 Operadors i mida del veïnatge

L'experiment 1 estudia l'efecte del conjunt d'operadors amb Hill Climbing, solució inicial `VORAZ` i l'escenari base. Amb només `MOVER`, `MOVER_ASIGNAR` o `MOVER_INTERCAMBIAR`, el benefici mitjà és el mateix: aproximadament 1.207.656 €. En canvi, `MOVER_ASIGNAR_CENTRALES`, que incorpora moviments d'obertura i tancament de centrals, arriba aproximadament a 1.358.883 €.

La comparació rèplica a rèplica entre `MOVER_ASIGNAR_CENTRALES` i `MOVER_ASIGNAR` dona 10 victòries de 10 i $p = 0{,}002$. El conjunt `COMPLETO` obté el mateix benefici mitjà que els operadors compostos, però amb més temps de còmput. Per tant, els operadors que permeten modificar directament l'estat de les centrals són els que produeixen el canvi més gran en el benefici observat.

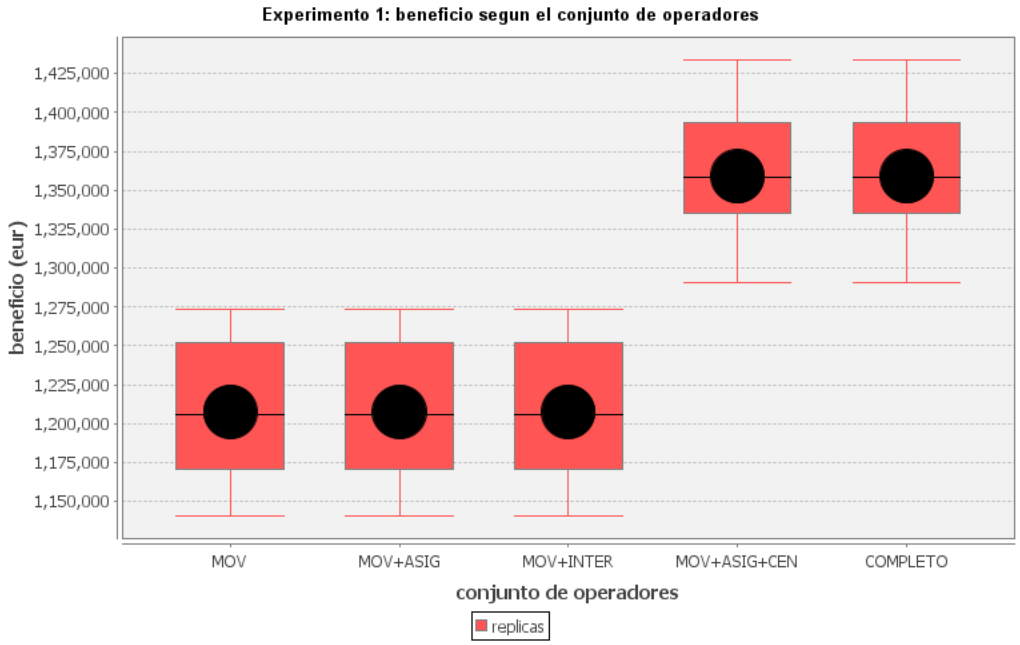


Figura 8.1: Benefici segons el conjunt d'operadors.

Amb el conjunt d'operadors compostos, el benefici és similar per als valors de k provats. El temps i el nombre de successors, però, augmenten quan el veïnatge és més gran. El valor $k = 8$ ofereix un compromís entre qualitat i cost de còmput en l'escenari base.

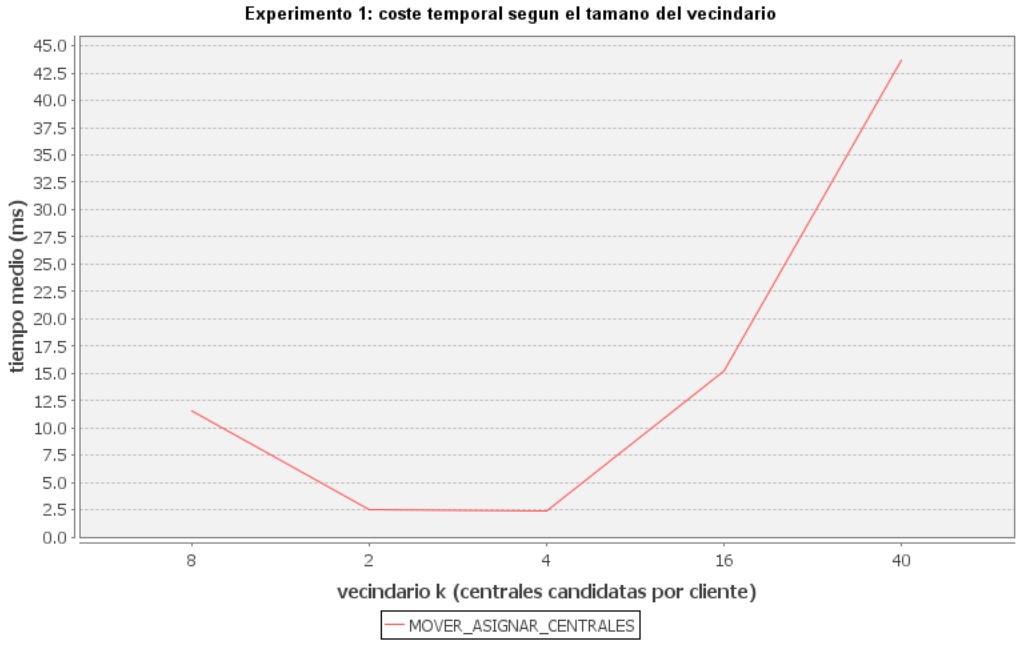


Figura 8.2: Efecte de la mida del veïnatge.

8.2 Estratègia de solució inicial

L'experiment 2 compara les inicialitzacions `ALEATORIA` i `VORAZ` mantenint fixos els operadors seleccionats a l'experiment anterior. La solució voraç parteix d'un benefici inicial molt superior i també acaba amb un benefici final més alt: aproximadament 1.358.883 € davant d'1.282.558 € de la inicialització aleatòria.

La solució `VORAZ` guanya en les 10 rèpliques, amb una diferència mitjana aproximada de 76.325 € i $p = 0{,}002$. La millora respecte de la solució inicial també és diferent: la inicialització aleatòria permet més recorregut durant la cerca, però no compensa la qualitat del punt de partida voraç.

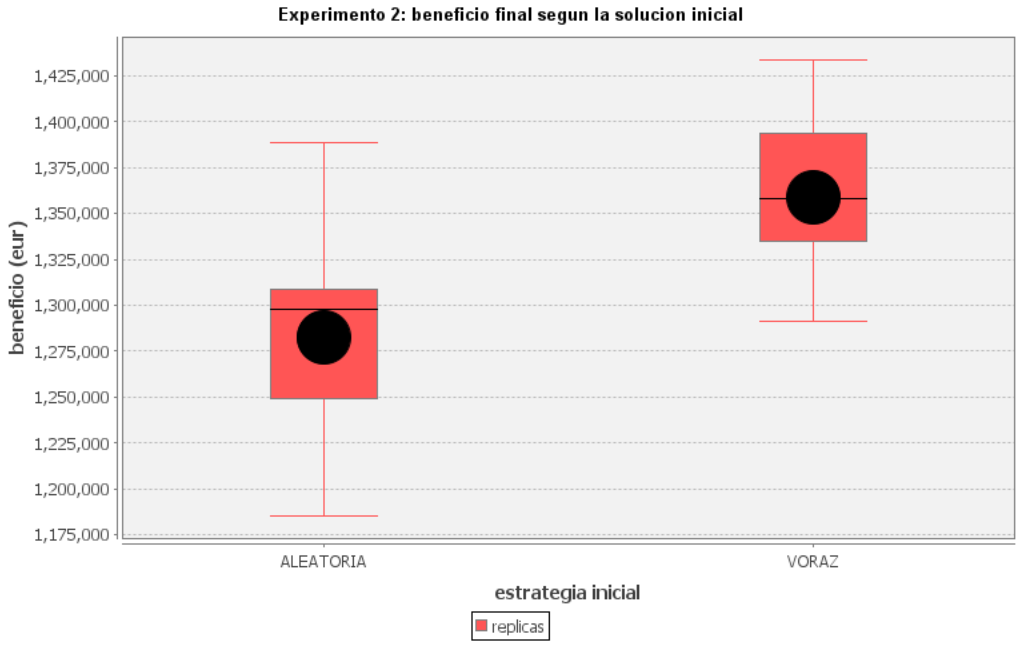


Figura 8.3: Benefici de les solucions inicials.

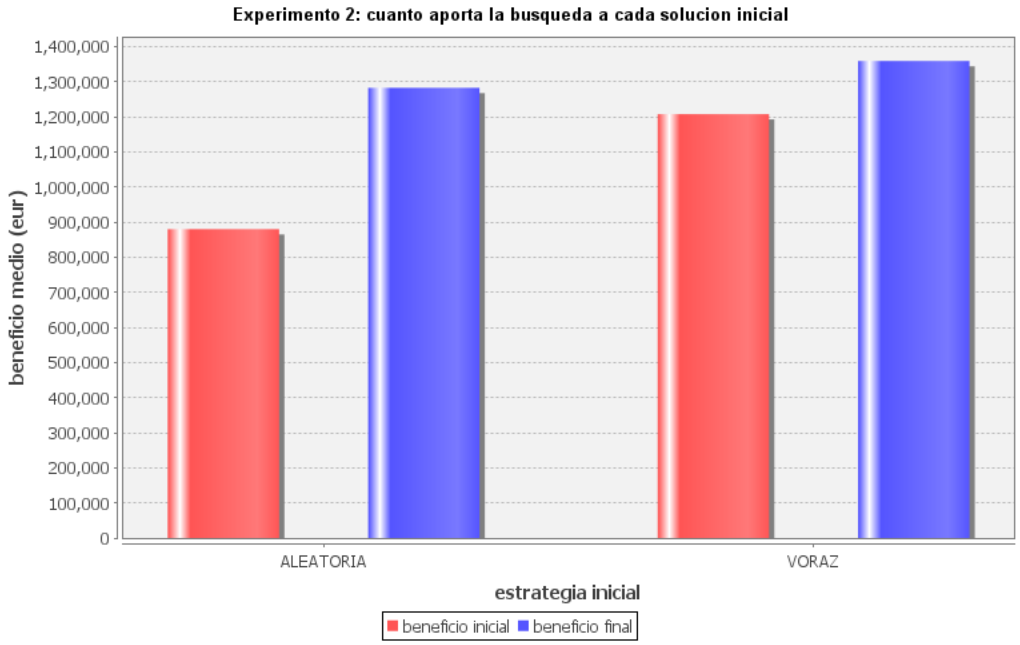


Figura 8.4: Millora obtinguda a partir de cada solució inicial.

8.3 Paràmetres de Simulated Annealing

L'experiment 3 estudia la mida del veïnatge i el paràmetre de refredament de Simulated Annealing, i posteriorment compara diferents nombres d'iteracions. Els valors de λ petits mantenen una temperatura efectiva durant més temps i comporten més temps de còmput. En

el barrido inicial, les diferències de benefici són petites en relació amb la variabilitat entre rèpliques.

Per als experiments posteriors es fixa una configuració de 100.000 iteracions, 1.000 iteracions per temperatura, $k = 5$ i $\lambda = 0{,}01$. Aquesta configuració no es tria només pel màxim benefici puntual del barrido, sinó perquè ofereix una qualitat similar a valors més lents amb un cost temporal menor.

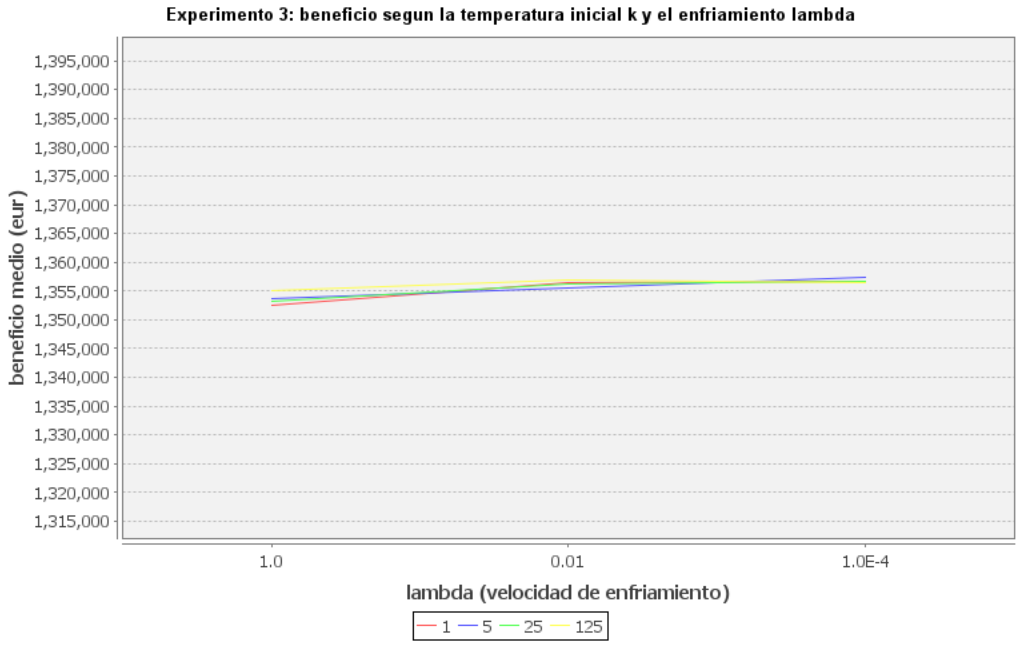


Figura 8.5: Benefici segons la temperatura i el veïnatge.

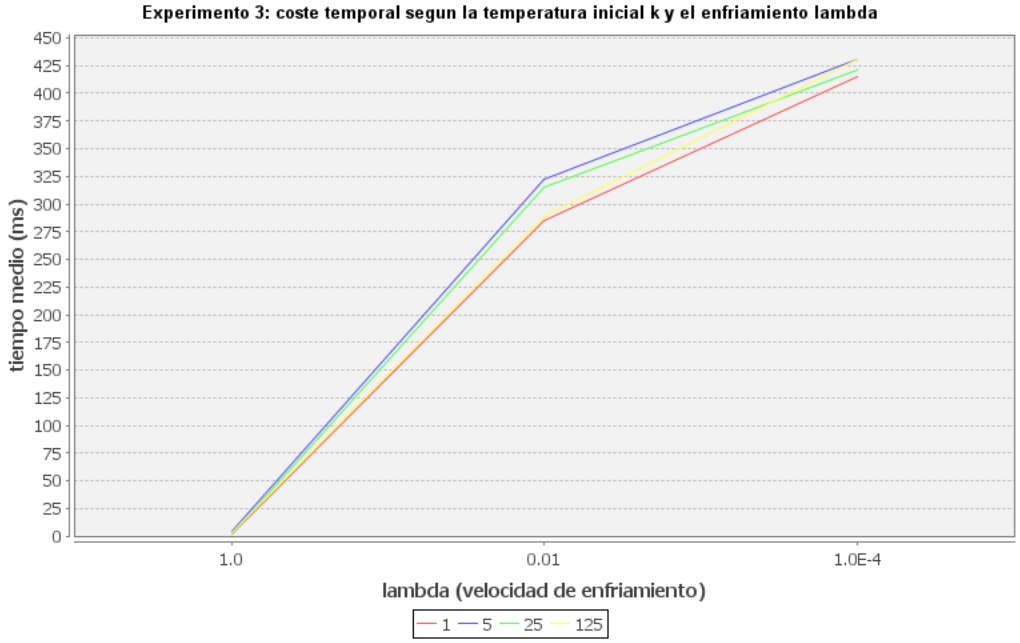


Figura 8.6: Temps de còmput segons la temperatura i el veïnatge.

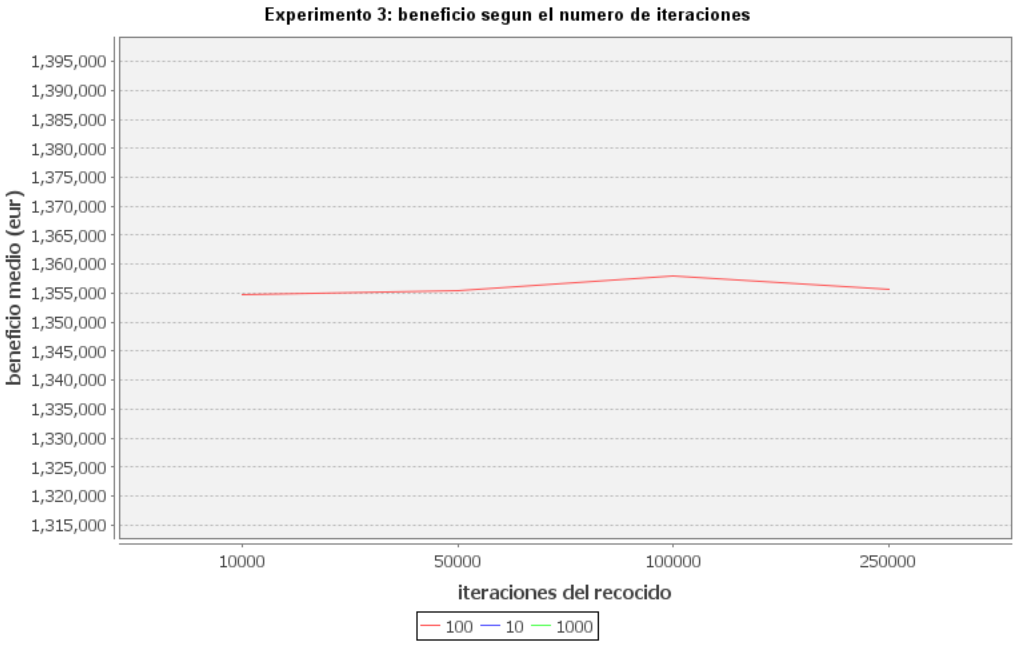


Figura 8.7: Efecte del nombre d'iteracions.

8.4 Escalabilitat amb el nombre de clients i centrals

L'experiment 4 mesura com varia el temps de còmput quan augmenta el nombre de clients o el nombre de centrals. Amb el nombre de centrals fix, el temps creix aproximadament amb el nombre de clients, perquè els operadors han de considerar més assignacions possibles. Quan augmenta el nombre de centrals, també augmenta el nombre d'alternatives que s'han de comprovar.

Aquest experiment mostra que el cost computacional depèn tant de la mida de l'assignació com de la mida del veïnatge. Per això el benefici no és l'única mètrica necessària per valorar una configuració.

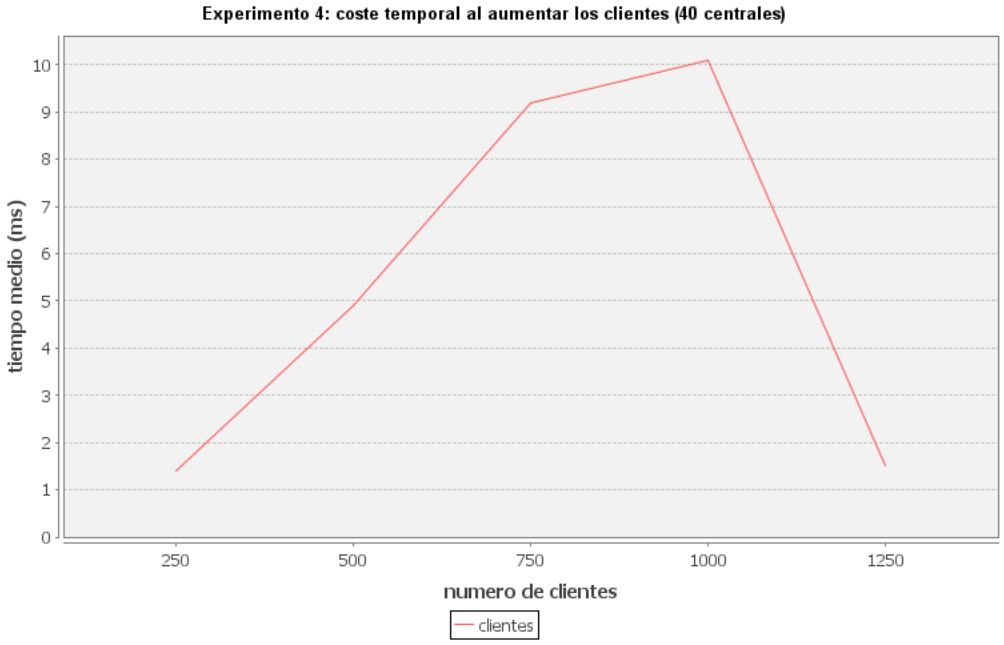


Figura 8.8: Temps segons el nombre de clients.

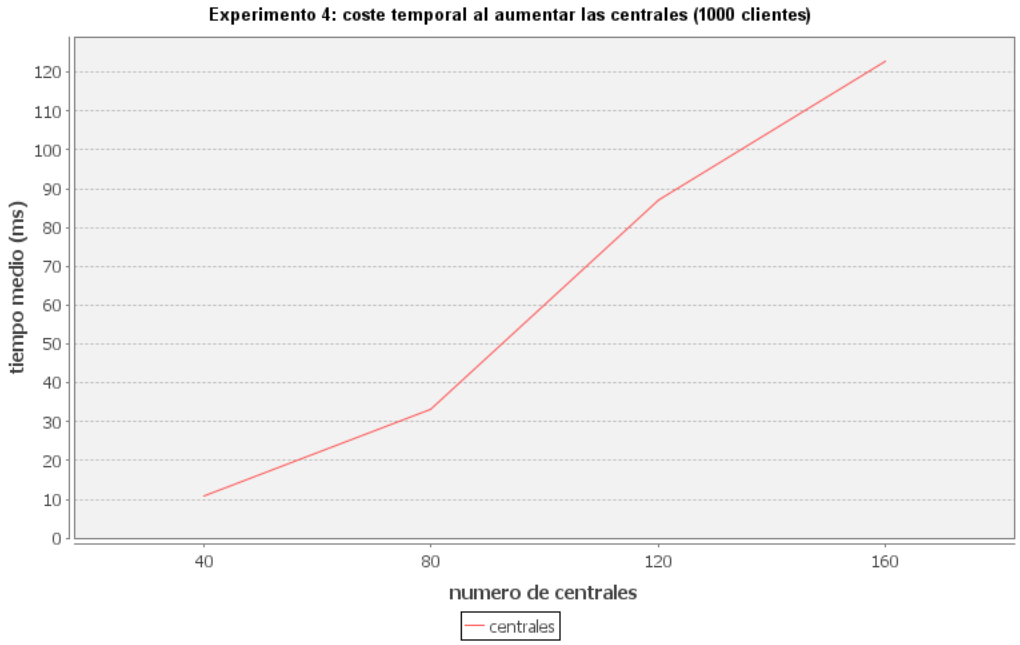


Figura 8.9: Temps segons el nombre de centrals.

8.5 Penalització dels clients garantits no servits

L'experiment 5 estudia el tractament de la restricció dels clients garantits. En lloc d'impedir sempre que quedin sense servir, la configuració experimental permet aquests estats i n'incorpora una penalització a la funció de qualitat. Això serveix per observar l'efecte de convertir una restricció dura en una penalització, però no substitueix la configuració normal del problema, en què els clients garantits s'han de servir.

Els resultats permeten comparar el benefici amb el nombre de clients garantits no servits i comprovar si la penalització és prou gran per evitar solucions invàlides. La validesa s'ha d'interpretar sempre separadament del benefici: una solució amb més benefici però que incompleix el servei garantit no és equivalent a una solució factible.

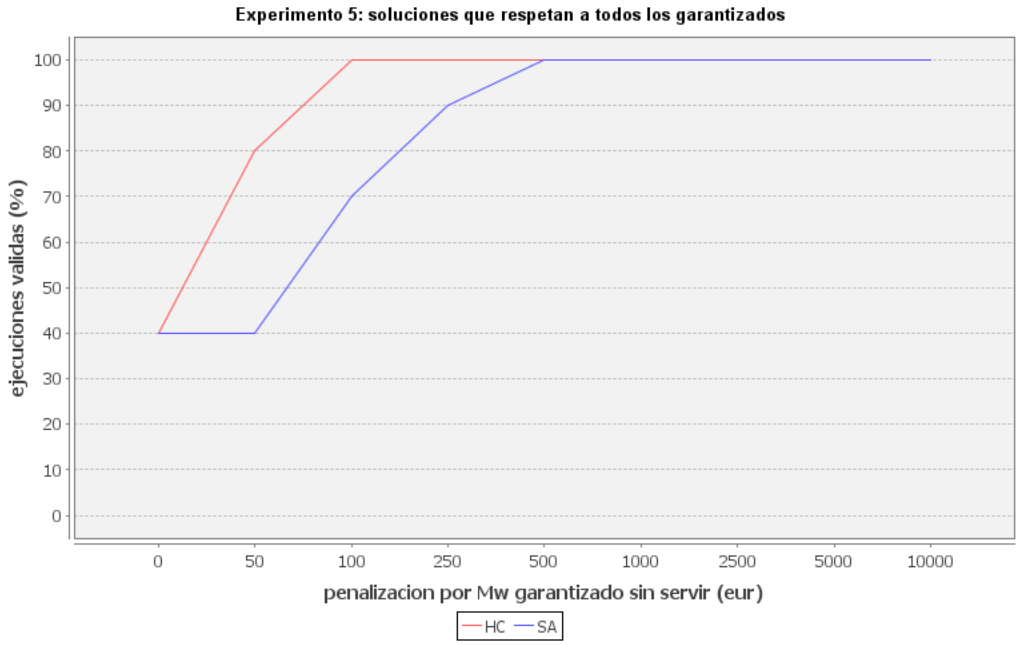


Figura 8.10: Efecte de la penalització dels clients garantits.

8.6 Ampliació del parc de centrals

L'experiment 6 varia el nombre de centrals de tipus C mantenint la demanda fixa. Afegir centrals no implica necessàriament obtenir un benefici més alt: una central només aporta capacitat quan és útil, però té un cost quan queda encesa. Per tant, el nombre de centrals disponibles i el nombre de centrals que finalment s'activen són magnituds diferents.

La figura mostra com canvien les centrals enceses i el benefici quan es duplica o triplica el parc de centrals de tipus C.

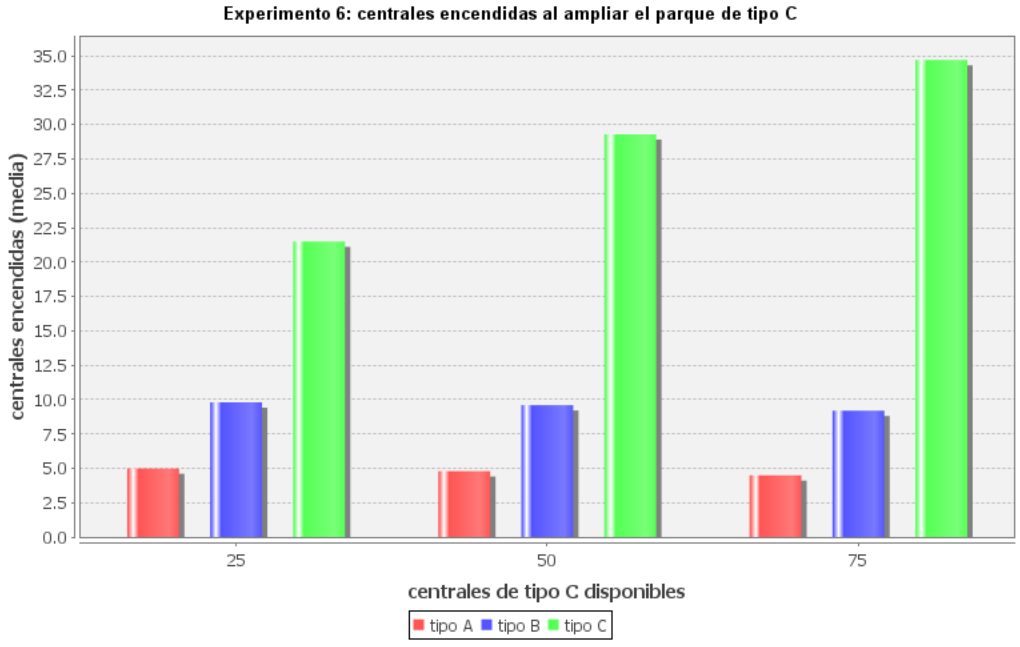


Figura 8.11: Centrals enceses en ampliar el parc de centrals de tipus C.

8.7 Comparació de les heurístiques

L'experiment 7 compara H1, H2 i H3 amb els operadors simples i compostos. Amb operadors simples, H2 pot distingir moviments que tenen el mateix benefici però generen pèrdues de transport diferents. H3 introdueix un criteri relacionat amb la capacitat ociosa i pot afavorir moviments que preparen el tancament d'una central.

Amb pes intermedi ($w = 10$), H3 guanya a H1 en les 10 rèpliques quan s'utilitzen operadors simples ($p = 0{,}002$). En canvi, la diferència entre H2 i H1 no és significativa ($p = 0{,}0625$). Amb operadors compostos, les diferències entre les heurístiques no són significatives en les comparacions disponibles.

Aquest patró és coherent amb el fet que els operadors compostos ja poden expressar directament els canvis d'obrir i tancar centrals. Les figures també permeten veure com el pes del criteri secundari afecta el benefici real i el nombre de passos de Hill Climbing.

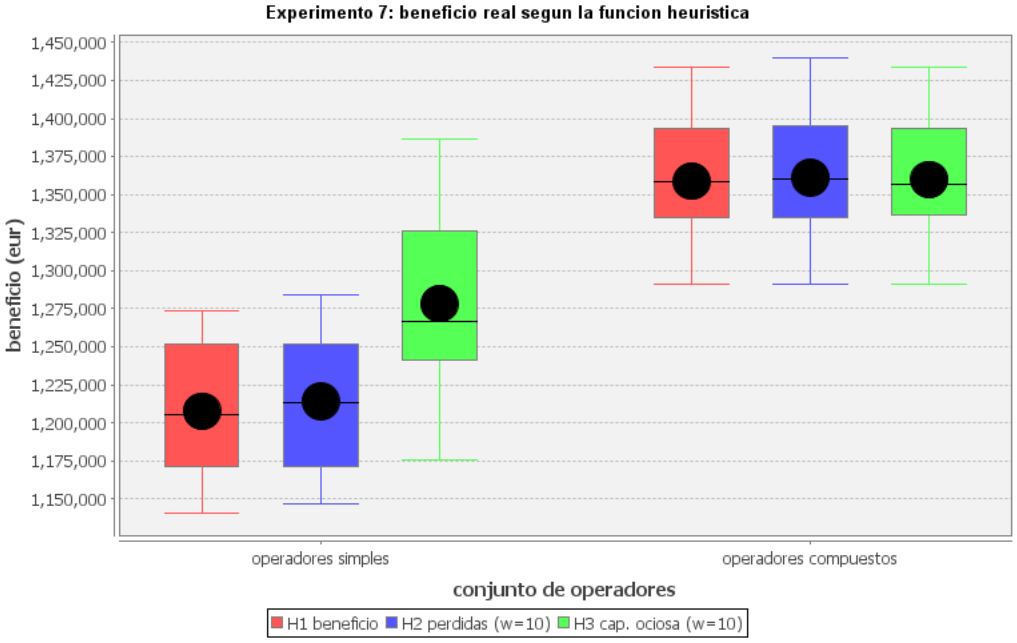


Figura 8.12: Benefici real segons la funció heurística.

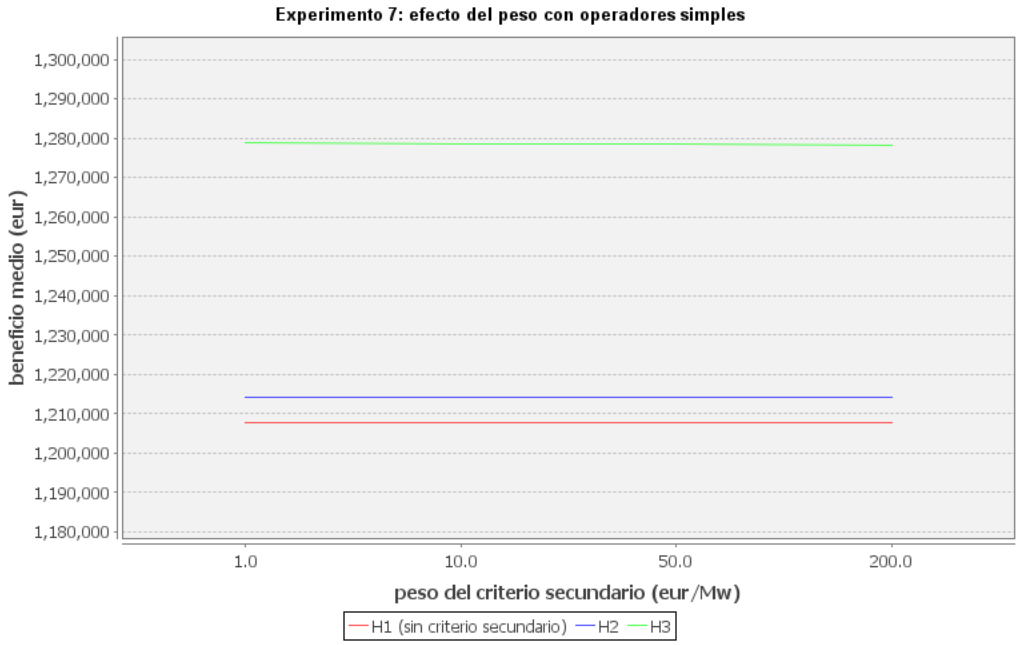


Figura 8.13: Efecte del pes amb operadors simples.

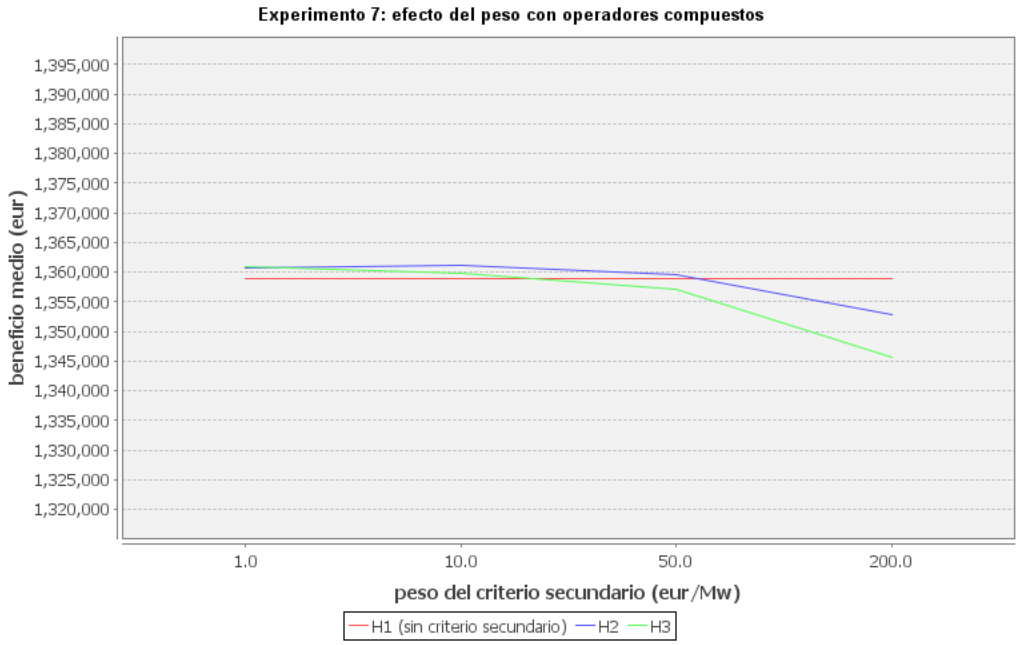


Figura 8.14: Efecte del pes amb operadors compostos.

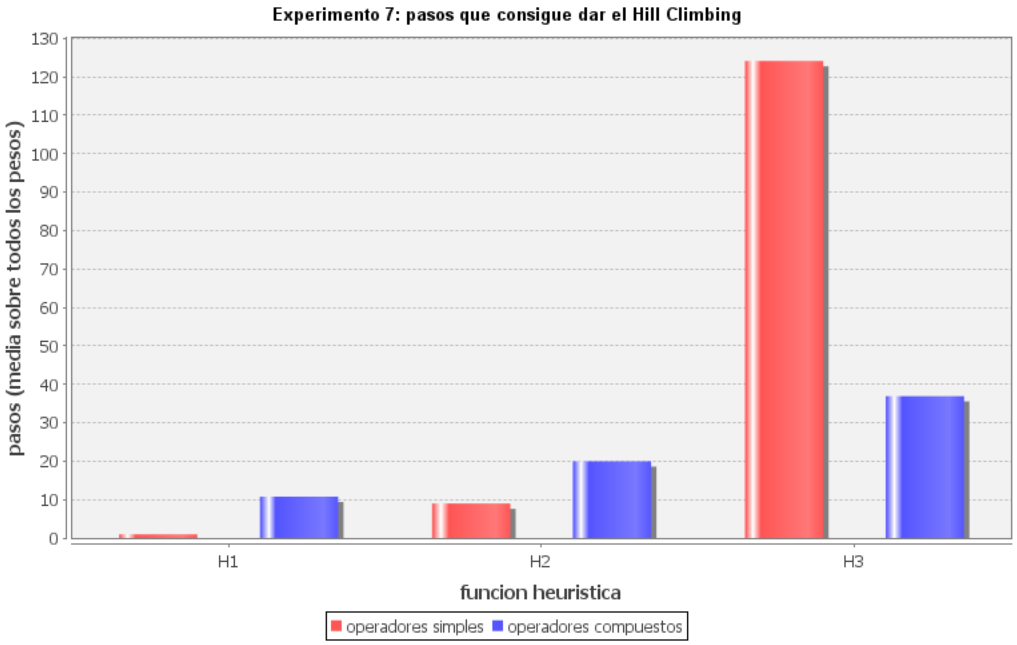


Figura 8.15: Passos de Hill Climbing segons la heurística.

8.8 Hill Climbing i Simulated Annealing

L'experiment 8 compara els dos algoritmes amb operadors simples i compostos. Amb `MOVER_ASIGNAR`, Simulated Annealing guanya a Hill Climbing en les 10 rèpliques, amb una diferència mitjana d'aproximadament 97.411 € i $p = 0{,}002$. Això és compatible amb la capacitat de Simulated Annealing de recórrer moviments laterals o acceptar temporalment moviments que no milloren el benefici.

Amb `MOVER_ASIGNAR_CENTRALES`, Simulated Annealing només guanya en 3 de les 10 rèpliques i la diferència no és significativa ($p = 0{,}3438$). En aquest cas els dos algoritmes obtenen beneficis similars, però Hill Climbing necessita menys temps de còmput.

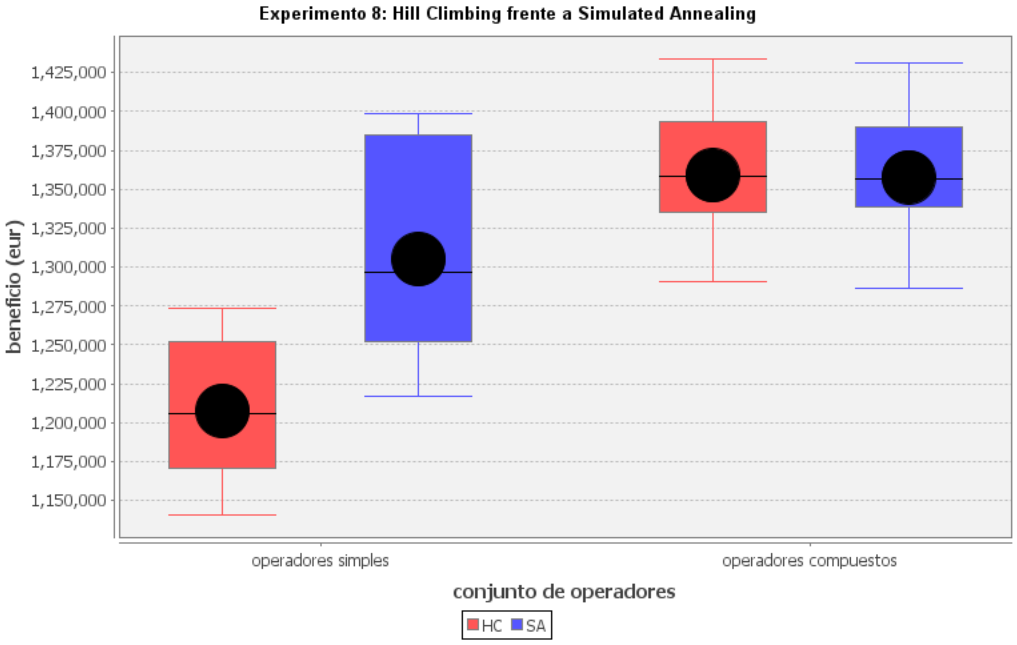


Figura 8.16: Comparació entre Hill Climbing i Simulated Annealing.

Capítol 9

Execució i reproducció

Aquest apartat descriu el procediment complet per repetir la compilació, les proves, els experiments i la generació dels resultats del report.

9.1 Requisits i directori de treball

Cal disposar de:

- **JDK 26 o superior**;
- les biblioteques `CentralEnergia.jar`, `AIMA.jar` i les dependències de `lib/lib/`;
- **Quarto** i una instal · lació de LaTeX (en cas de voler generar de nou el PDF).

Totes les ordres següents s'executen des de l'**arrel del repositori**. Els fitxers CSV i les figures es desen a `results/`, mentre que les classes compilades es desen a `out/`.

9.2 Compilació

Per compilar el codi propi i deixar les classes compilades a `out/`:

```
javac -cp "lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  -d out (Get-ChildItem -Recurse src -Filter *.java | % FullName)
```

El classpath inclou les dues biblioteques principals. Les dependències addicionals necessàries per a les gràfiques es carreguen durant l'execució mitjançant `lib\lib\*`.

9.3 Comprovació de la representació

Abans d'executar la bateria completa, es pot comprovar la coherència de l'estat, les inicialitzacions i les operacions:

```
java -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.PruebaEstado
```

També es pot executar una comprovació ràpida de la bateria:

```
java -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.MainExperimentos smoke
```

9.4 Execució dels experiments

Per executar tots els experiments amb les configuracions encadenades:

```
java -Xmx2g -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.MainExperimentos todos
```

També es pot executar un experiment concret indicant el seu identificador:

```
java -Xmx2g -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.MainExperimentos 1
```

Els identificadors disponibles són del `1` al `8`. També es poden utilitzar els àlies `smoke`, `heuristicas` i `comparativa` per a les execucions corresponents.

Quan s'executa `todos`, els experiments es llancen en ordre i les eleccions realitzades als experiments inicials es reutilitzen en els següents. Per això, per reproduir exactament la bateria completa, és preferible executar `MainExperimentos todos` en una única invocació.

Cada experiment escriu el detall de les rèpliques a `results/expN.csv`. L'execució també mostra per terminal els resums de les mitjanes i les desviacions típiques.

9.5 Generació de figures i comparacions

Un cop creats els CSV, es generen les figures:

```
java -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.Graficas
```

Les figures es desen a `results/` amb noms com `fig1a-operadores.png` i `fig8-hc-vs-sa.png`.

Per executar les comparacions rèplica a rèplica i obtenir els contrastos binomials:

```
java -cp "out;lib\CentralEnergia.jar;lib\AIMA.jar;lib\lib\*" `
  IA.PracticaEnergia.Comparaciones
```

9.6 Generació del report

Finalment, des de l'arrel del repositori, es genera el document amb:

```
quarto render report\report.qmd
```

El PDF resultant es desa al directori `report/`. Perquè les figures apareguin correctament, cal haver executat prèviament `Graficas` i conservar la carpeta `results/` en la ubicació esperada.

Capítol 10

Anàlisi i discussió

Els experiments permeten analitzar separadament els principals factors que afecten la cerca: la forma del veïnatge, la qualitat de l'estat inicial, la funció heurística i l'algoritme utilitzat. Les comparacions s'han de llegir conjuntament amb el temps de còmput i amb la validesa de les solucions, perquè un benefici més alt no és suficient si s'obté incomplint una restricció.

10.1 Importància dels operadors

El resultat més clar és que els operadors que permeten obrir i tancar centrals tenen un efecte més gran que l'ampliació indiscriminada del veïnatge. Els operadors simples poden moure clients, però sovint no modifiquen el conjunt de centrals enceses i, per tant, no poden transformar directament una meseta en una millora de benefici. Els operadors compostos resolen aquesta limitació amb una sola transició.

El conjunt `COMPLETO` no millora el benefici respecte de `MOVER_ASIGNAR_CENTRALES` en les dades disponibles, però sí que augmenta el cost de la cerca. Això indica que afegir successors només és útil quan aporta moviments que l'algoritme no pot obtenir amb el conjunt compost.

10.2 Efecte de la solució inicial

La inicialització `VORAZ` és el factor més consistent entre les comparacions de l'estat inicial. Parteix d'un benefici més alt i acaba superant `ALEATORIA` en les 10 rèpliques comparades. La cerca continua sent necessària, però comença des d'una configuració que ja reutilitza millor les centrals enceses i la capacitat disponible.

Això mostra que, en un problema de cerca local amb mesetes, la inicialització no és només una qüestió de temps d'arrencada: condiciona quina zona de l'espai de solucions es pot explorar amb el nombre de moviments disponible.

10.3 Paper de les heurístiques

H1 és el criteri més directe perquè coincideix amb l'objectiu real. H2 i H3 aporten informació addicional quan els operadors simples generen moviments amb el mateix benefici. En particular, H3 obté una millora estadísticament significativa respecte d'H1 en aquesta configuració d'operadors, mentre que H2 no presenta una diferència significativa amb el pes comparat.

Quan s'utilitzen operadors compostos, les heurístiques secundàries deixen de ser tan necessàries: el moviment ja pot incloure el canvi d'activar o desactivar una central i H1 pot detectar

directament la millora. Això no significa que H2 i H3 siguin incorrectes, sinó que el seu avantatge depèn de la capacitat expressiva del veïnatge.

10.4 Comparació dels algoritmes

Simulated Annealing és útil quan Hill Climbing queda atrapat en una meseta formada per operadors simples. En aquest cas supera Hill Climbing en totes les rèpliques comparades. Amb operadors compostos, en canvi, els dos algoritmes obtenen resultats similars i Hill Climbing té un cost temporal inferior.

La conclusió no és que un algoritme sigui sempre millor que l'altre. La selecció depèn de si el veïnatge permet expressar les transicions importants: quan no les permet, el mecanisme d'acceptació de Simulated Annealing és útil; quan sí que les permet, l'exploració exhaustiva del veïnatge de Hill Climbing resulta més eficient.

10.5 Qualitat i cost computacional

Els resultats mostren un compromís entre benefici i temps. Augmentar k o el nombre d'iteracions pot incrementar el nombre d'estats examinats sense produir una millora proporcional del benefici. Igualment, disposar de més centrals no implica necessàriament un benefici més alt, perquè les centrals enceses tenen un cost.

Per tant, la configuració s'ha d'avaluar amb un conjunt de mètriques i no amb una única mitjana. El benefici mesura la qualitat econòmica, mentre que el temps i els nodes expandits mesuren el cost de trobar-la i les mètriques de servei comproven que la solució respecta les condicions del problema.

Capítol 11

Conclusions

La pràctica ha modelat la distribució d'energia com un problema de cerca local amb restriccions de capacitat, pèrdues de transport i clients amb servei garantit. La representació incremental de `EstadoEnergia` permet avaluar i actualitzar els successors sense recalcular tota l'assignació.

Les conclusions principals són:

1. **Els operadors compostos són determinants.** `MOVER_ASIGNAR_CENTRALES` obté un benefici clarament superior als operadors simples perquè pot obrir i tancar centrals en una mateixa transició.
2. **La solució inicial voraç és preferible.** `VORAZ` supera `ALEATORIA` en les rèpliques comparades i proporciona un punt de partida de més qualitat per a la cerca.
3. **Les heurístiques secundàries depenen del veïnatge.** H2 i H3 poden trencar mesetes quan els operadors són simples, però amb operadors compostos el benefici pur H1 acostuma a ser suficient.
4. **Simulated Annealing és útil en presència de mesetes.** Amb operadors simples pot superar Hill Climbing, però quan el veïnatge ja conté els moviments compostos necessaris, Hill Climbing aconsegueix una qualitat similar amb menys temps.
5. **La capacitat i les pèrdues condicionen la factibilitat.** Augmentar el nombre de centrals o el nombre de clients modifica tant l'espai de cerca com la possibilitat de servir tota la demanda, i no garanteix per si sol una millora del benefici.

En conjunt, la configuració més equilibrada per a l'escenari estudiat és la inicialització `VORAZ`, els operadors `MOVER_ASIGNAR_CENTRALES` i un veïnatge moderat, com $k = 8$ per a Hill Climbing. Aquesta recomanació combina el benefici observat amb un cost de còmput inferior al del conjunt complet. Es tracta d'una conclusió per a l'escenari i els paràmetres estudiats, no d'una garantia d'optimalitat global.

Capítol 12

Treball d'innovació

12.1 Tema escollit

12.2 Repartiment del treball

Qui s'ha encarregat de buscar la informació per a cada apartat del document:

Apartat del treball	Responsable de la cerca d'informació

12.3 Referències trobades

Referència	Apartats per als quals és rellevant	Data d'accés

12.4 Dificultats trobades